

2.3 The Shape of a Matrix



Core skills you'll master in this section:

- ▶ The uses of column vectors and row vectors in data analysis and machine learning.
- ▶ A tall-narrow matrix represents many samples with few features, while a flat matrix represents the opposite; the two can be converted into each other via transpose.
- ▶ Square matrices: a prerequisite for many operations, such as the determinant, matrix inverse, and eigenvalue decomposition.
- ▶ Symmetric matrices: a symmetric matrix's transpose equals itself.
- ▶ Diagonal matrices: off-diagonal elements are zero; can be square or rectangular.
- ▶ Identity matrix: a square diagonal matrix whose main-diagonal elements are all 1, especially important in matrix multiplication.
- ▶ Triangular matrices: upper triangular and lower triangular matrices are transposes of each other, simplifying the solving of equations.
- ▶ Block matrices: splitting a large matrix into submatrices to improve the readability and efficiency of complex operations.

As mentioned in the first section of this chapter, matrices come in a rich variety of shapes. This section describes the various matrix shapes in detail — each of these special shapes has its own unique uses. A matrix's shape is the cornerstone of matrix operations: it determines not only whether basic operations such as addition, subtraction, and matrix multiplication can be carried out between matrices, but also profoundly affects the result of an operation and its underlying geometric meaning. Moreover, many matrix operations impose requirements on matrix shape. For example, the determinant, LU decomposition, Cholesky decomposition, eigenvalue decomposition, the trace, and so on, all require a square matrix as a precondition.

In addition, in data analysis, machine learning, and scientific computing, matrices of different shapes — whether row vectors, column vectors, tall-narrow matrices, flat matrices, or square matrices — each play a unique role. And if we're not satisfied with a matrix's shape, we can slice it into a shape we like; the tool for this is the block matrix, which is also what this section covers.

Figure 1 shows various common matrix shapes. This section will introduce these common matrix shapes and their uses in detail.

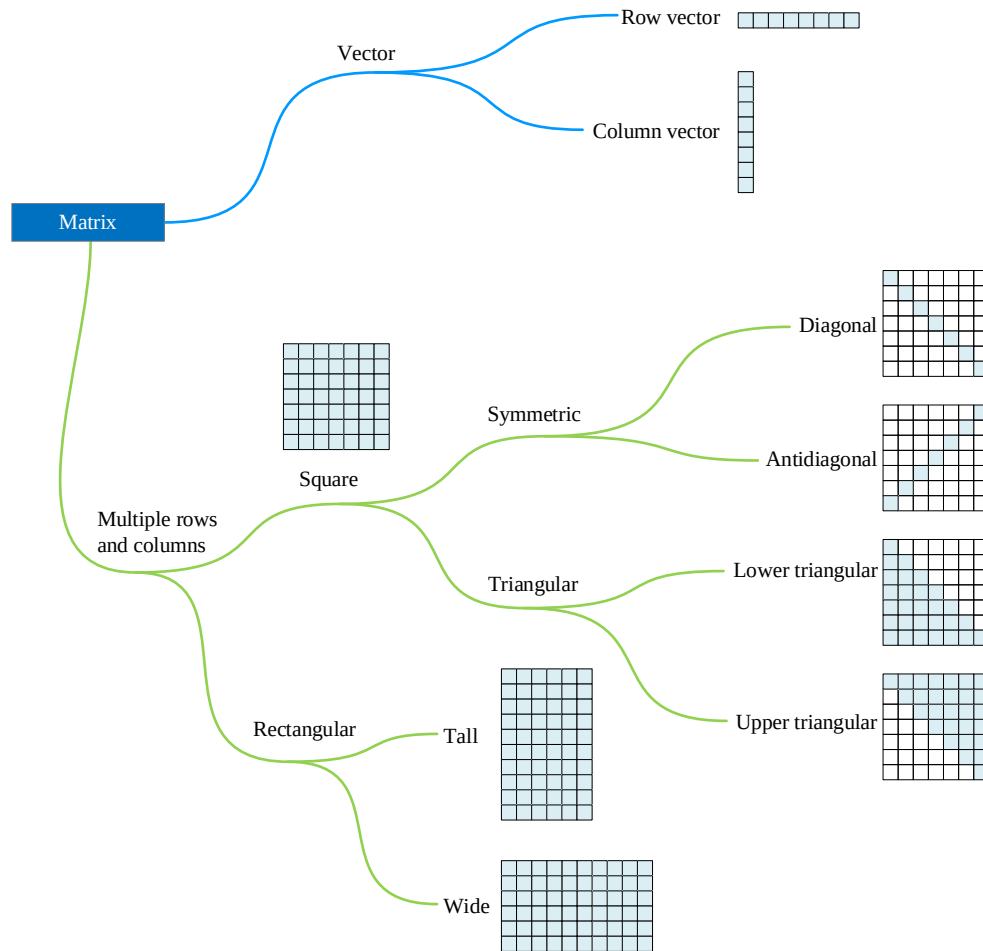


Figure 1. Several Common Matrix Shapes

Column Vectors

The row vectors and column vectors introduced earlier in this book are also matrices of a special shape.

A row vector can be viewed as a matrix with one row and multiple columns, while a column vector is a matrix with one column and multiple rows. The two convert into each other via the transpose operation: transposing a row vector gives a column vector, and transposing a column vector gives a row vector.

Although row vectors and column vectors are simple in form, they play an indispensable role in data analysis, linear algebra, and machine learning. Let's start by talking about column vectors.

Any point in the plane can be written as a 2D column vector, for example

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \end{bmatrix}$$

A point in 3D space can be written as a 3D column vector, for example

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix}$$

As you're already familiar with, in RGB space, color vectors can all be written in the form of column vectors. For example, the red, green, and blue vectors can be written as

$$\mathbf{e}_1 = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \quad \mathbf{e}_2 = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}, \quad \mathbf{e}_3 = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

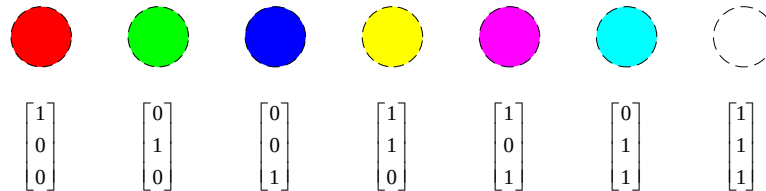


Figure 2. Representing Colors with Column Vectors

In a data matrix, column vectors are often used to store the data for a single feature.

For example, in the iris dataset, each column vector represents one feature, such as sepal length, sepal width, petal length, or petal width. Taking sepal length as an example, its column vector is

$$\begin{bmatrix} 5.1 \\ 4.9 \\ 4.7 \\ \vdots \end{bmatrix}$$

This column vector records the sepal length of every sample in the dataset. From a data perspective, a column vector represents the distribution of one feature across the dataset, and based on it we can perform various statistical computations, such as computing the mean, variance, or plotting a distribution histogram.

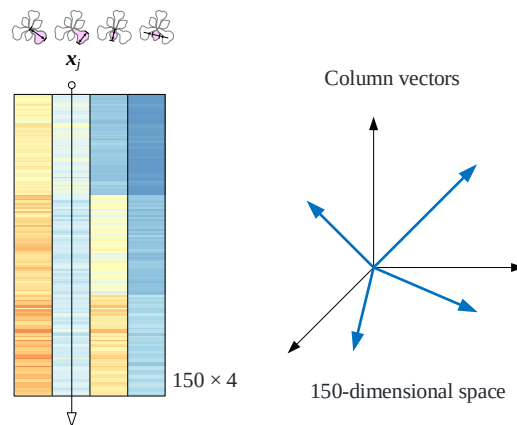


Figure 3. Iris Data Column Vector, Excluding the Label Column

From a feature-engineering perspective, the column-vector form makes it convenient to perform operations such as standardization, normalization, or feature extraction, laying the foundation for subsequent machine learning modeling.

Unless stated otherwise, vectors mentioned in this book default to column vectors. Column vectors are often the objects that linear transformations act on. Simply put, a linear transformation is

a transformation that preserves the rules of vector addition and scalar multiplication, such as rotation, scaling, and reflection.

And to understand linear transformations, we first need to become comfortable with matrix multiplication, which is the topic of the next section.

Row Vectors

In data analysis, row vectors are typically used to represent a single sample point in a dataset.

Specifically, each row of data corresponds to one sample, and its columns represent that sample's different features.

For example, in the classic iris dataset, each row corresponds to the record of one iris flower and can be represented as a row vector. For example, the row vector for the first row

$$\mathbf{x}^{(1)} = [5.1 \quad 3.5 \quad 1.4 \quad 0.2 \quad \text{Setosa}]$$

The values above represent, respectively, one iris sample's sepal length, sepal width, petal length, petal width, and class label.

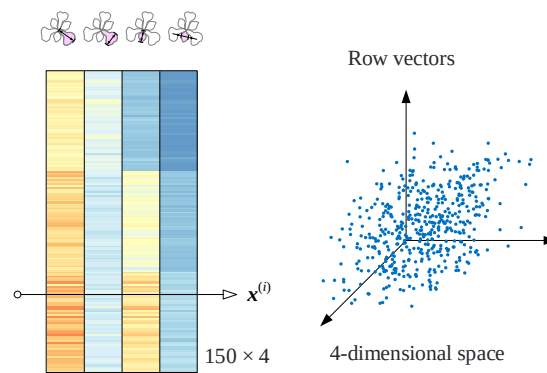


Figure 4. Iris Data Row Vector, Excluding the Label Column

The row-vector form lets us reduce complex, multidimensional data to a single compact mathematical object, making it easier to store, process, and model.

Taking color as an example, Figure 5 contains a set of n colors, which together trace out the RGB color cube's outer frame. We can store these color vectors in a matrix, where each row vector of the matrix represents one color.

This matrix is then a rectangular matrix.

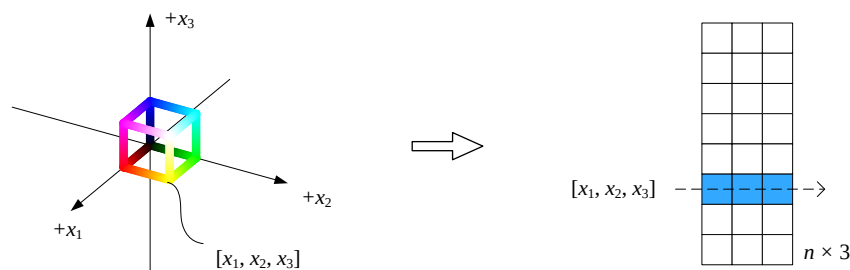


Figure 5. The RGB Color Cube's Outer Frame

Rectangular Matrices

A rectangular matrix is a matrix whose number of rows and number of columns are not equal; rectangular matrices come in two forms – tall-narrow (tall) and flat (wide).

A tall-narrow matrix is a matrix whose number of rows is far greater than its number of columns. This kind of matrix is shaped like a tall, standing column – the iris dataset is one example.

Tall-narrow matrices are often used to represent datasets where the number of samples greatly exceeds the number of features. For example, in machine learning, a dataset with 1000 samples, each having 10 features, can be represented as a 1000×10 tall-narrow matrix.

A flat matrix is a matrix whose number of columns is far greater than its number of rows. Flat matrices are often used to represent datasets where the number of features greatly exceeds the number of samples. For example, in gene expression data analysis, a dataset with 100 samples, each having 10,000 gene features, can be represented as a $100 \times 10,000$ flat matrix.

Of course, transposing a tall-narrow matrix gives a flat matrix.

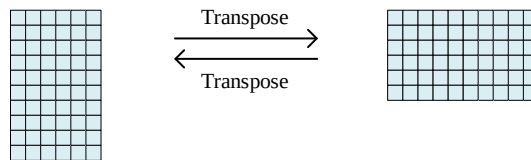


Figure 6. Tall-Narrow and Flat Matrices Convert into Each Other via Transpose

Square Matrices

A square matrix is a special and important form within the matrix family, whose number of rows equals its number of columns – that is, its shape is $n \times n$.

An $n \times n$ square matrix A can be written as:

$$A = \begin{bmatrix} a_{1,1} & a_{1,2} & \cdots & a_{1,n} \\ a_{2,1} & a_{2,2} & \cdots & a_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n,1} & a_{n,2} & \cdots & a_{n,n} \end{bmatrix}$$

For a square matrix, the main diagonal is the line running from the upper-left element to the lower-right element; this diagonal neatly divides the matrix into the diagonal itself, plus an upper and a lower triangular region.

If we fill in the omitted elements with 0, we can see the different types of square matrices, which is what the rest of this section covers.

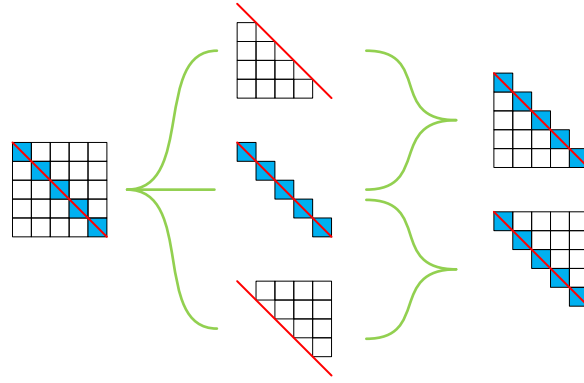


Figure 7. A Square Matrix's Main Diagonal Divides It into Different Regions

Table 1 shows matrix operations that require a square matrix as a precondition. You don't need to worry about these linear algebra concepts right now — we suggest coming back to review this table after finishing the book.

Table 1. Matrix Operations That Require a Square Matrix

Operation	Description
Matrix determinant	The determinant is defined only for square matrices; it is used to determine important properties such as whether a matrix is invertible (non-singular).
Trace of a matrix	The trace of a matrix is the sum of a square matrix's main-diagonal elements; it reflects the overall "scaling effect" of the matrix's linear transformation, or the sum of its eigenvalues.
Matrix inverse	The inverse exists only when the matrix is square and its determinant is nonzero. A non-square matrix can instead have a pseudo-inverse.
Matrix power	An integer power of a matrix, such as A^n , generally requires the matrix to be square — otherwise the matrix multiplication cannot be carried out.
Eigenvalue decomposition	Eigenvalue decomposition applies only to square matrices.
Spectral decomposition	Spectral decomposition applies only to symmetric square matrices.
Diagonalization	Only a square matrix can be diagonalized — that is, converted into a diagonal matrix via a similarity transformation.
LU decomposition	Decomposes a square matrix into a lower triangular matrix and an upper triangular matrix.
Cholesky decomposition	A positive-definite matrix (which must first be a symmetric square matrix) can undergo Cholesky decomposition.
Quadratic form	A quadratic form also requires a square matrix as a precondition.
Rayleigh quotient	The matrix multiplication in the Rayleigh quotient's numerator likewise requires a square matrix.

Symmetric Matrices

As mentioned in the previous section, if an $n \times n$ square matrix A 's transpose equals itself, that is,

$$A = A^T$$

then $\mathbf{A}$ is called a symmetric matrix.

This property means that the elements of square matrix $\mathbf{A}$ are mirror-symmetric about the main diagonal.

Specifically, in matrix $\mathbf{A}$, the element a_{ij} located at row i , column j , equals the element a_{ji} located at row j , column i . This symmetry means that a symmetric matrix can be fully described just by storing the main diagonal and the elements on one side of it (the upper or lower triangle).

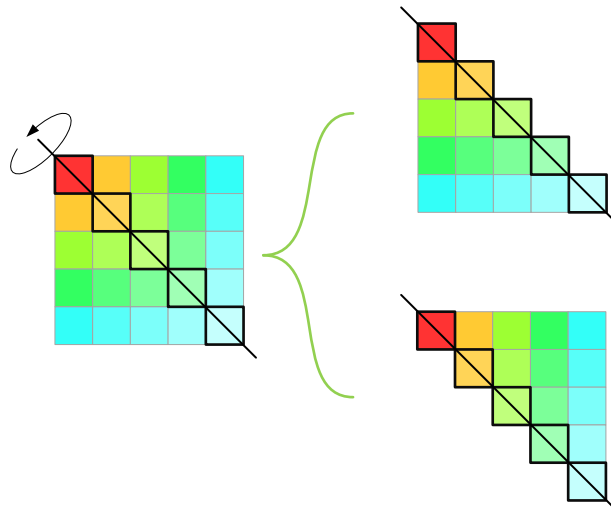


Figure 8. Decomposing a Symmetric Matrix

Diagonal Matrices

In a diagonal matrix, all nonzero elements lie on the main diagonal, and every other off-diagonal element must be 0.

⚠ Note that a diagonal matrix's main-diagonal elements can be either 0 or nonzero.

There are two kinds of diagonal matrices: the square diagonal matrix and the rectangular diagonal matrix.

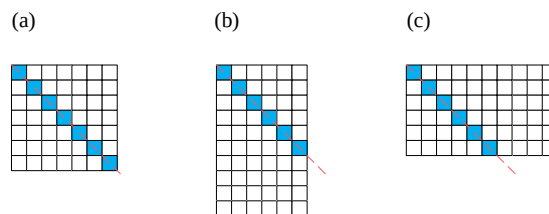


Figure 9. Diagonal Matrices of Different Shapes

A square diagonal matrix has the following form

$$\mathbf{D} = \begin{bmatrix} d_1 & 0 & \cdots & 0 \\ 0 & d_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & d_n \end{bmatrix}$$

If the number of rows is greater than the number of columns (tall-narrow), a rectangular diagonal matrix has the form

$$\mathbf{D}_{m \times p} = \begin{bmatrix} d_1 & 0 & \cdots & 0 \\ 0 & d_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & d_p \\ 0 & 0 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 0 \end{bmatrix} \quad (m > p)$$

If the number of rows is less than the number of columns (flat), a rectangular diagonal matrix has the form

$$\mathbf{D}_{m \times p} = \begin{bmatrix} d_1 & 0 & \cdots & 0 & 0 & \cdots & 0 \\ 0 & d_2 & \cdots & 0 & 0 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & d_m & 0 & \cdots & 0 \end{bmatrix} \quad (m < p)$$

Identity Matrix

The identity matrix is a special square diagonal matrix. All the elements on the identity matrix's main diagonal are 1, and every other element is 0.

In this book, the identity matrix is denoted $\mathbf{I}$.

$$\mathbf{I}_{n \times n} = \begin{bmatrix} 1 & 0 & \cdots & 0 \\ 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 1 \end{bmatrix}$$

For example,

$$\mathbf{I}_{2 \times 2} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad \mathbf{I}_{3 \times 3} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Many other references also use $\mathbf{E}$ to represent the identity matrix. In this book, $\mathbf{E}$ is reserved specifically to represent the standard orthonormal basis.

Each column of the identity matrix represents a standard orthonormal basis vector, for example

$$\mathbf{I}_{2 \times 2} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \quad \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} \mathbf{e}_1^{(2)} & \mathbf{e}_2^{(2)} \end{bmatrix}, \quad \mathbf{I}_{3 \times 3} = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} \quad \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} \quad \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} \mathbf{e}_1^{(3)} & \mathbf{e}_2^{(3)} & \mathbf{e}_3^{(3)} \end{bmatrix}$$

The identity matrix plays the role of “the number 1” in linear algebra — it is the identity element for matrix multiplication. You'll soon see the identity matrix's role in matrix multiplication and matrix inverses.

Triangular Matrices

A triangular matrix is also a special kind of square matrix.

If all the elements below a square matrix's diagonal are zero, the matrix is called an upper triangular matrix:

$$U_{n \times n} = \begin{bmatrix} u_{1,1} & u_{1,2} & \dots & u_{1,n} \\ 0 & u_{2,2} & \dots & u_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & u_{n,n} \end{bmatrix}$$

If all the elements above a square matrix's diagonal are zero, the matrix is called a lower triangular matrix:

$$L_{n \times n} = \begin{bmatrix} l_{1,1} & 0 & \dots & 0 \\ l_{2,1} & l_{2,2} & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ l_{n,1} & l_{n,2} & \dots & l_{n,n} \end{bmatrix}$$

Clearly, transposing an upper triangular matrix gives a lower triangular matrix, and vice versa.

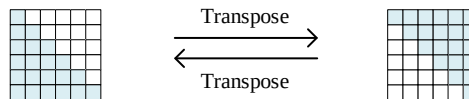


Figure 10. Transposing an Upper Triangular Matrix Gives a Lower Triangular Matrix, and Vice Versa

Block Matrices

If you're “not satisfied” with a matrix's shape, we can “cut” the matrix into a shape you're happy with!

As mentioned earlier in this chapter, a block matrix splits a large matrix into several smaller matrices, much like cutting a block of tofu; each smaller matrix is called a block, or submatrix.

For example, given the following 5×5 matrix A

$$A = \begin{bmatrix} 3 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

We can slice this matrix once, top and bottom; matrix A is split into two matrices

$$\mathbf{A} = \begin{bmatrix} 3 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 1 \\ \hline 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Let's give these two submatrices names

$$\mathbf{A} = \begin{bmatrix} \mathbf{A}_1 \\ \mathbf{A}_2 \end{bmatrix}$$

We can see that $\mathbf{A}_1$ is a square diagonal matrix of shape 3×3 , while $\mathbf{A}_2$ is a zero matrix.

As another example, the following matrix $\mathbf{A}$ is sliced once horizontally and once vertically; $\mathbf{A}$ is split into four submatrices

$$\mathbf{A} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ \hline 0 & 0 & 1 & 2 & 3 \\ 0 & 0 & 4 & 5 & 5 \\ 0 & 0 & 7 & 8 & 9 \end{bmatrix}$$

Let's give these four submatrices names

$$\mathbf{A} = \begin{bmatrix} \mathbf{A}_{1,1} & \mathbf{A}_{1,2} \\ \mathbf{A}_{2,1} & \mathbf{A}_{2,2} \end{bmatrix}$$

You can see that this matrix has been split into four submatrices; these four submatrices become the matrix's new "elements," that is,

$$\mathbf{A}_{1,1} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad \mathbf{A}_{1,2} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}, \quad \mathbf{A}_{2,1} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 \end{bmatrix}, \quad \mathbf{A}_{2,2} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 8 \end{bmatrix}$$

You may have already noticed a drawback of this notation: it doesn't directly tell us the shape of each submatrix. We need to further specify the number of rows and columns of each submatrix.

Figure 11, Figure 12, and Figure 13 illustrate this block-matrix notation, modeled after NumPy matrix indexing and slicing. The difference is that all three figures here use 1-based counting.

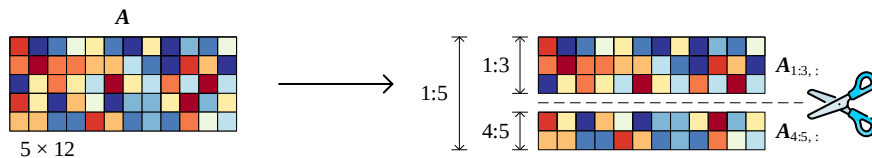


Figure 11. Slicing Once, Top and Bottom

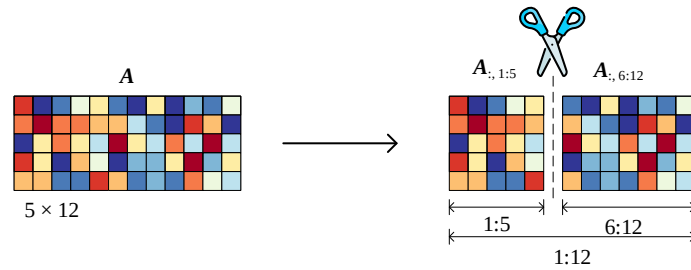


Figure 12. Slicing Once, Left and Right

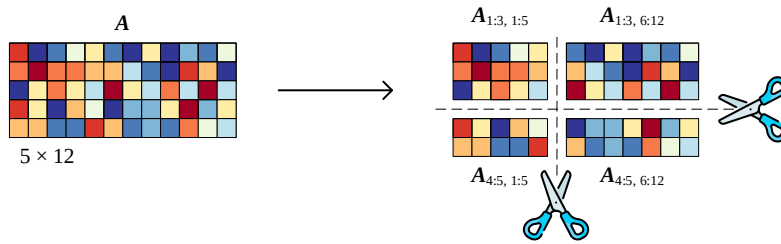


Figure 13. Slicing Once Horizontally and Once Vertically

Block matrices let us decompose a complex, large matrix into several smaller matrices, making computation more efficient — especially when working with large-scale data or performing matrix operations. Later in this book, we'll use block matrices to help us understand matrix multiplication.



Please use tools such as DeepSeek/ChatGPT to complete the following exercises for this section.

Q1. Use `numpy.eye()` to generate a 3×3 identity matrix, and check that any two of its columns are orthogonal (their inner product is 0).

Q2. Write your own code (without using `numpy.diag()`) to extract the main-diagonal elements of an arbitrary matrix.

Q3. Write your own code (without using `numpy.diag()`) to turn a given sequence of main-diagonal elements into a square diagonal matrix.

Q4. Learn to use `numpy.triu()` to extract the upper triangular part of a square matrix.

Q5. Learn to use `numpy.tril()` to extract the lower triangular part of a square matrix.

Q6. Based on whether a matrix's transpose equals the original matrix, write code to determine whether the matrix is symmetric. Use `numpy.array_equal()` to check whether two matrices are equal.

Q7. Use `numpy.random.randint()` to generate a 5×12 matrix, and slice it into submatrices following Figure 11, Figure 12, and Figure 13.